Orin-PS-PL 协同通用开发流程

本文说明在 HLS IP 核导出之后，如何在 Vivado 中为 Zynq UltraScale+ MPSoC 搭建 Block Design、选择 FPGA 器件/板卡配置、连接 PS 和 PL 侧 IP、导出硬件平台，并在 Vitis IDE 中创建、编译、下载和运行工程。
当前目标板器件：

```
xczu9eg-ffvb1156-2-i
```

1. 工程中 Orin、PS、PL 怎样配合

HLS C/C++ 代码经过 Vitis HLS 综合后，会变成 PL 里的 RTL/IP 外设。Orin 看到的不是 C 函数，而是一个远端算子服务或硬件加速端点。
协同关系如下：

```
Orin

  运行 Qwen2.5 Python/CUDA 推理主流程
  负责准备 tensor、转换 BF16 数据、发送算子请求、接收 FPGA 输出

AXU9EGB / Zynq UltraScale+ PS
  运行 bare-metal 或 lwIP 程序
  负责 Ethernet/TCP 通信、DDR buffer 管理、cache 维护
  通过 AXI-Lite 配置 PL 侧 HLS IP 控制寄存器

AXU9EGB / Zynq UltraScale+ PL
  放置 HLS 导出的算子 IP
  通过 AXI master 从 PS DDR 读取输入/权重
  计算后通过 AXI master 把输出写回 PS DDR
```

以RMSNorm 为例，端到端数据流是：

```
Orin Qwen2.5 runtime
  |
  | 1. 取出 RMSNorm hidden_states 和 weight
  | 2. 转成约定的 BF16 little-endian payload
  | 3. 通过 Ethernet/TCP 发给 AXU9EGB PS
  v
Zynq UltraScale+ PS bare-metal/lwIP server
```

```
  |
  | 4. 接收 TCP payload
  | 5. 把 input/weight 拷贝到 DDR buffer
  | 6. Flush DCache，确保 PL 能读到新数据
  | 7. 通过 AXI-Lite 写 HLS IP 寄存器：
  |      input_addr / weight_addr / output_addr / eps / ap_start
  v
PL HLS IP
  |
  | 8. m_axi_gmem0 从 DDR 读 input
  | 9. m_axi_gmem1 从 DDR 读 weight
  | 10. m_axi_gmem2 向 DDR 写 output
  v
Zynq UltraScale+ PS
  |
  | 11. 轮询 ap_done 或处理中断
  | 12. Invalidate DCache，读取 output buffer
  | 13. 通过 TCP 返回 Orin
  v
Orin Qwen2.5 runtime
  |
  | 14. 把 FPGA 输出转回 tensor
  | 15. 继续后续推理层
```

所以整个系统有两条关键通道：

```
控制通道：
PS CPU -> AXI-Lite -> HLS IP control register
数据通道：
PS DDR <-> AXI HP/HPC port <-> PL HLS IP m_axi port
```

2. 需要确认的内容

2.1 HLS IP 已经导出

HLS IP 已经通过 Vitis HLS 导出为 Vivado IP Catalog 格式。一般目录类似：

```
hls/<module_name>/<hls_project>/solution1/impl/ip
```

2.2 记录 IP 接口

每次导入一个新 HLS IP 前，都要先确认它有哪些端口。以当前 RMSNorm IP 为例：

```
s_axi_control
  AXI-Lite slave，PS 用来配置寄存器和启动 IP
m_axi_gmem0
  AXI master，IP 读取 input buffer
m_axi_gmem1
  AXI master，IP 读取 weight buffer
m_axi_gmem2
  AXI master，IP 写 output buffer
ap_clk
  PL 时钟
ap_rst_n
  低有效复位
```

当前 RMSNorm 顶层函数是：

```
void rmsnorm_layer0_prefill(
    const bf16_t input[RMSNORM_ELEMENTS],
    const bf16_t weight[RMSNORM_HIDDEN_SIZE],
    bf16_t output[RMSNORM_ELEMENTS],
    float eps
);
```

当前固定数据规模：

```
seq_len     = 36
hidden_size = 896
input       = 36 * 896 * 2 bytes
weight      = 896 * 2 bytes
output      = 36 * 896 * 2 bytes
```

2.3 开发连接

```
JTAG       用于 Vitis 下载 bitstream 和 app
UART       用于看 bare-metal/lwIP 串口打印
Ethernet   用于 Orin <-> AXU9EGB PS 通信
电源       确认板卡供电稳定
```

串口配置：

```
115200 baud
8 data bits
no parity
```

```
1 stop bit
no flow control
```

3. Vivado 工程

3.1 选择板卡

在 `Parts` 页直接选择器件：

```
Family       Zynq UltraScale+ MPSoCs
Device       xczu9eg
Package      ffvb1156
Speed        -2
Temperature  i
Part         xczu9eg-ffvb1156-2-i
```

4. 添加 HLS IP Repository

Vivado GUI：

```
Tools
  -> Settings
  -> IP
  -> Repository
  -> Add
```

添加 HLS IP 导出目录，例如：

```
D:\master\yan\LLM\code\LLM-
FPGA\hls\rmsnorm_layer00_prefill\rmsnorm_layer0_prefill_prj\solution1\impl\ip
```

点击：

```
Apply
OK
```

然后在 IP Catalog 中搜索你的 IP 名称，例如：

```
rmsnorm_layer0_prefill
```

5. 创建 Block Design

Vivado 左侧 Flow Navigator：

```
IP Integrator
  -> Create Block Design
```

5.1 添加 Zynq UltraScale+ MPSoC

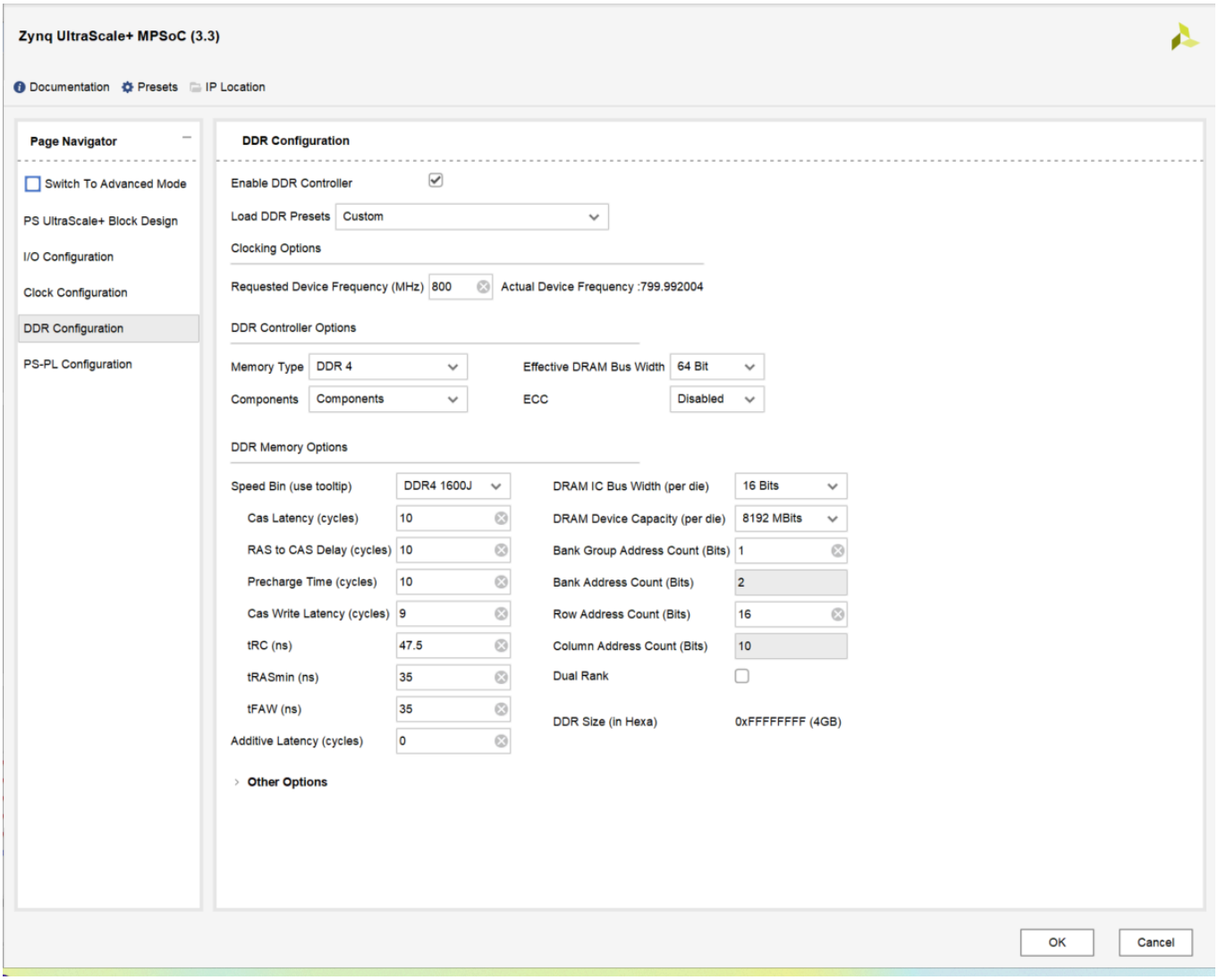
在 Block Design 空白处点击 `+`，添加：

```
Zynq UltraScale+ MPSoC
```

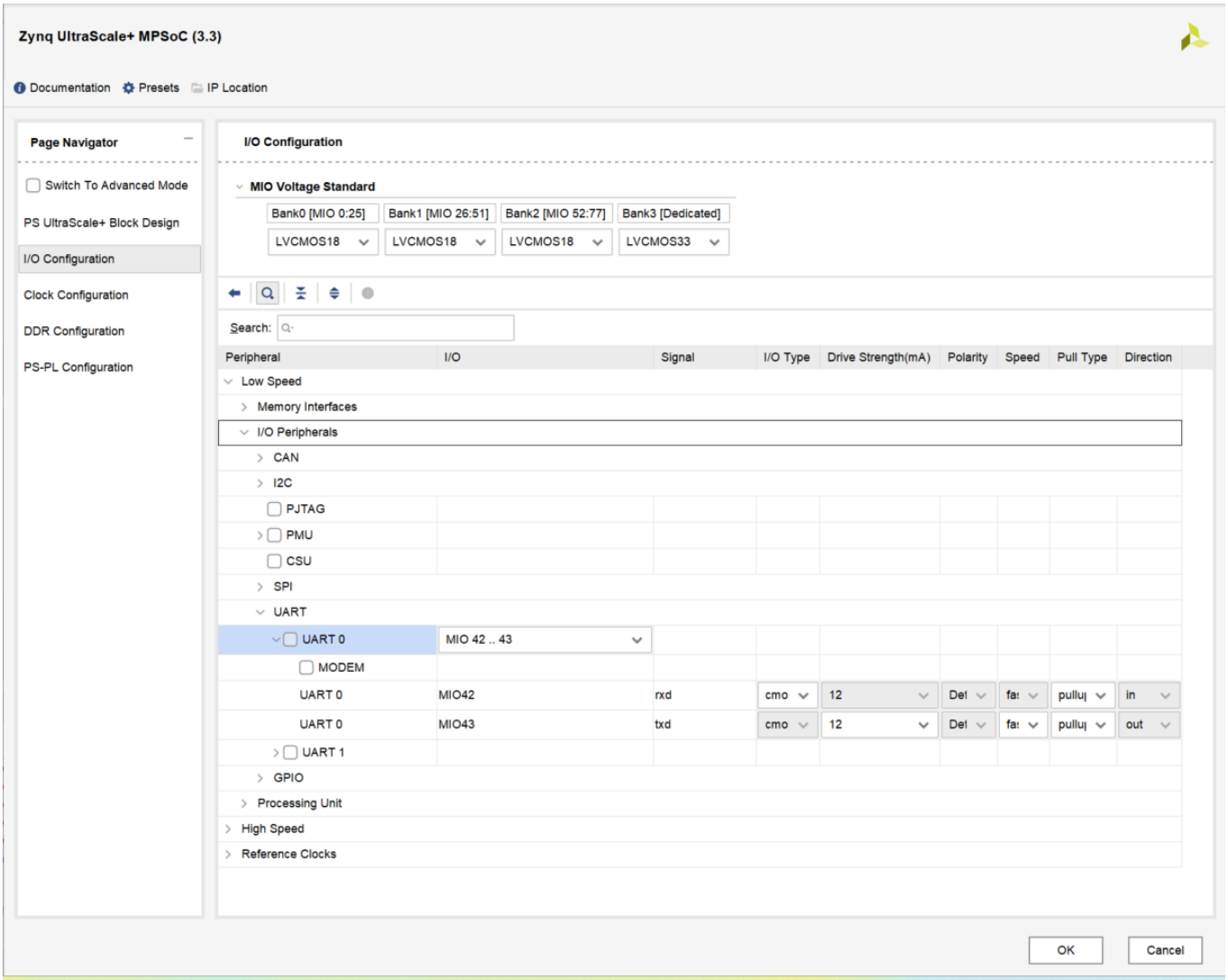
5.2 PS 配置重点

双击 `zynq_ultra_ps_e_0`，进入配置界面。

DDR



UART

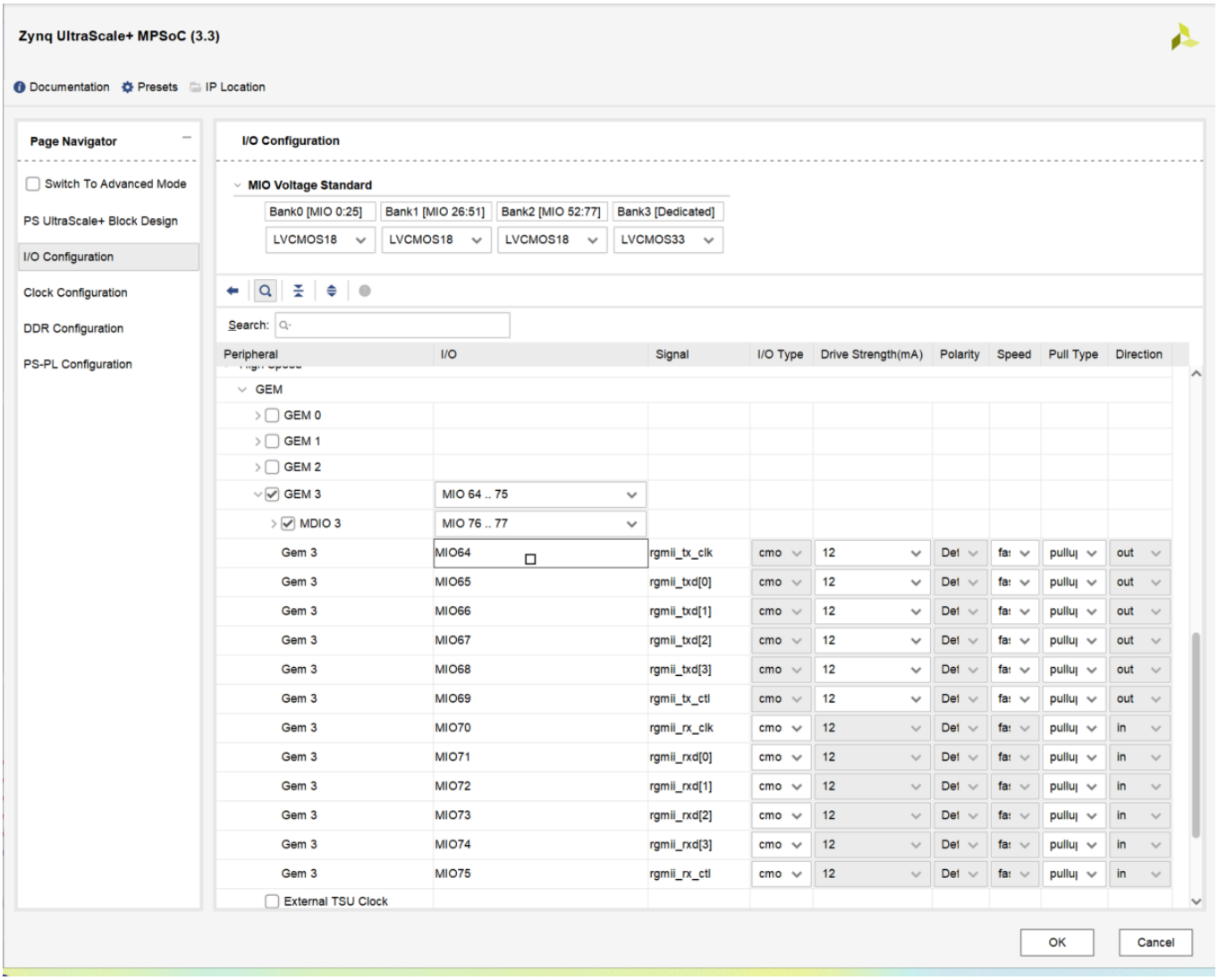


启用一个 PS UART 作为 Vitis standalone 的 stdout/stdin。后续 BSP 中也要把 stdin/stdout 指到这个 UART。

Ethernet

如果后续要跑 Orin <-> PS 的 lwIP server，启用板上连接到 RJ45/PHY 的 GEM。
当前 AXU9EGB 联调文档按 GEM3 检查：

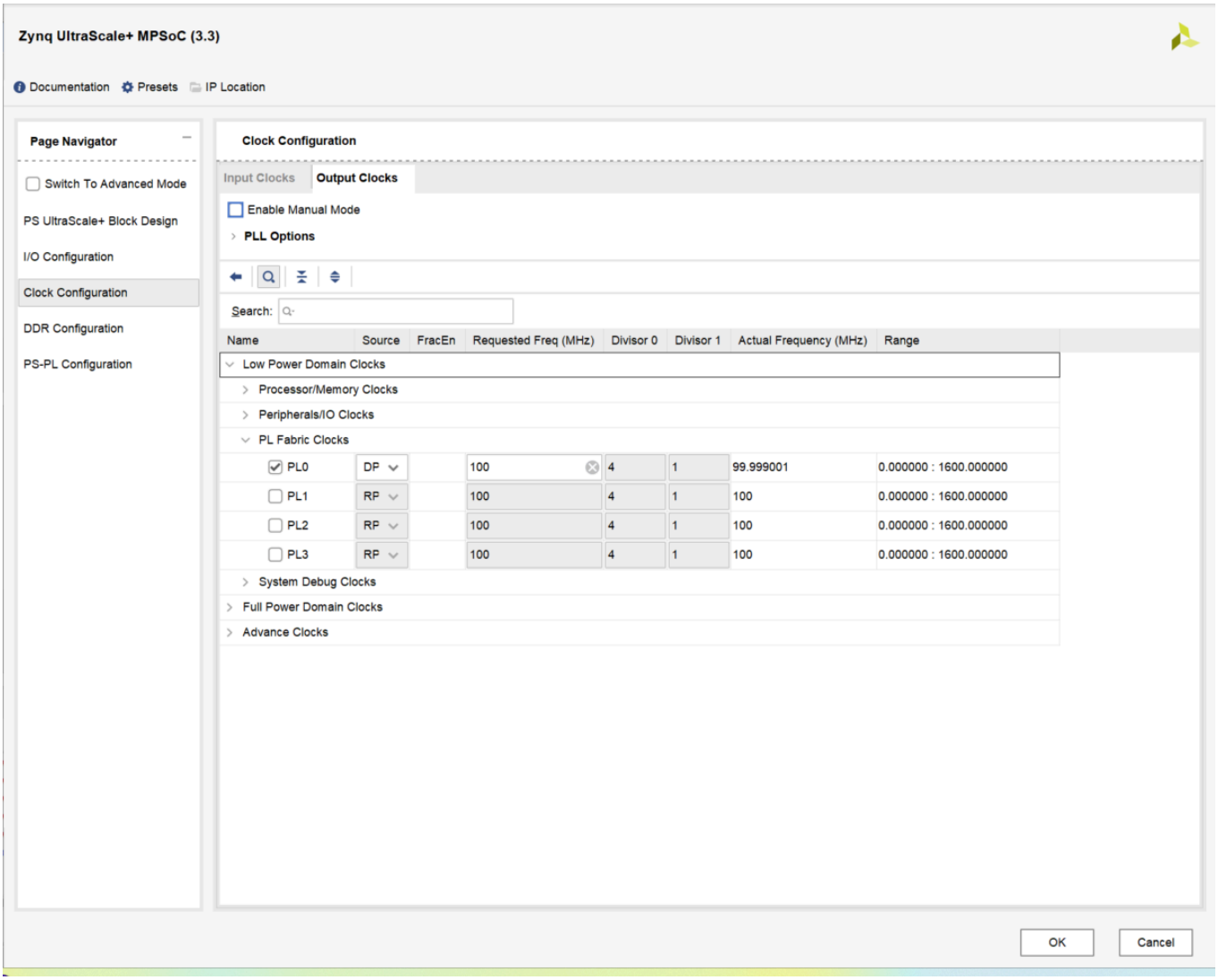
```
GEM3
MDIO
MIO64..75
MIO76..77
```



PL Fabric Clock

打开至少一个 PL 时钟：

```
PL0 clock / pl_clk0
```



PL Reset

启用：

```
pl_resetn0
```

PS 到 PL 的 AXI Master

开启：

```
M_AXI_HPM0_FPD
```

作用：

```
PS A53 通过这个 master 口访问 PL 中的 AXI-Lite 外设
也就是写 HLS IP 的 s_axi_control 寄存器
```

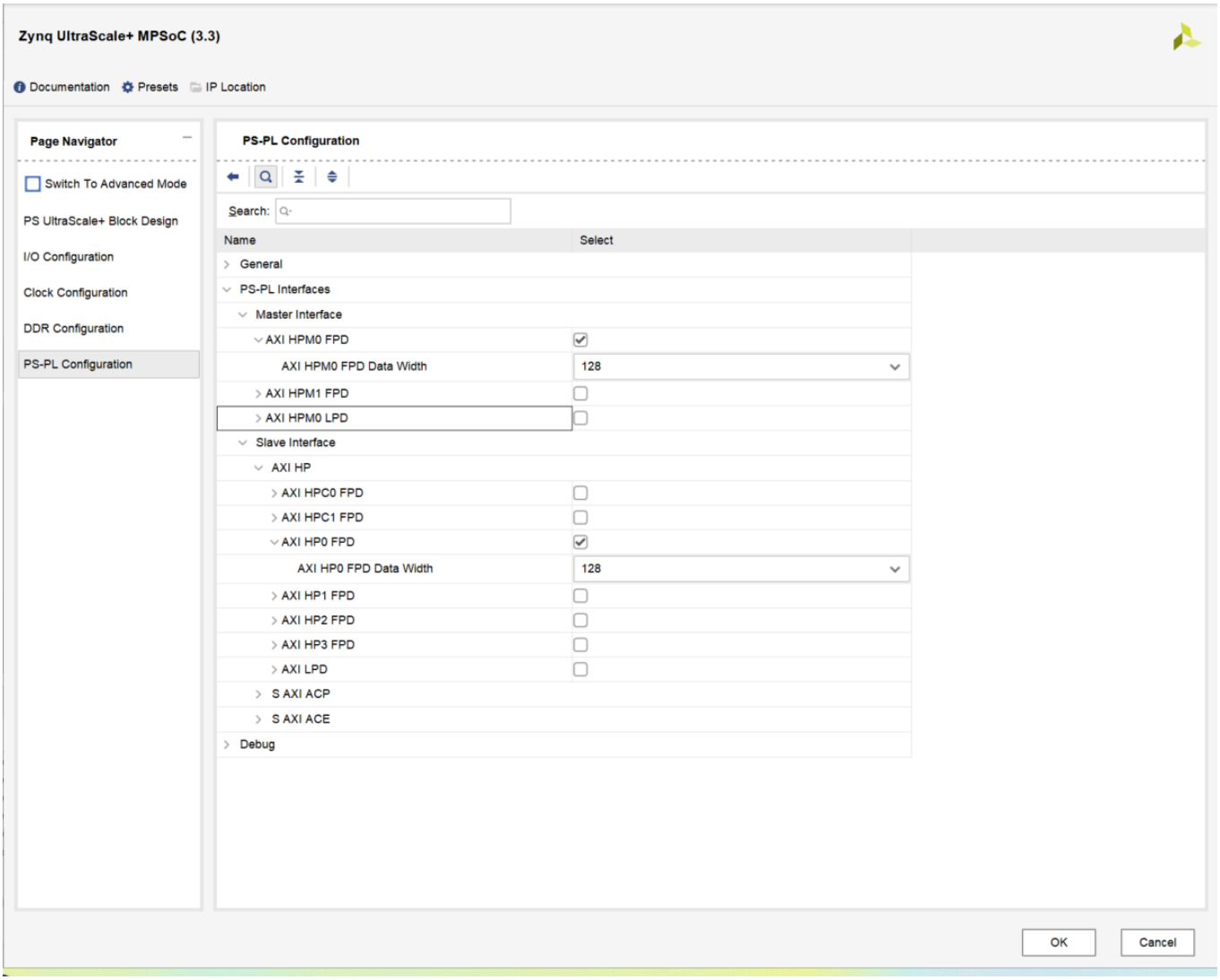
PL 到 PS DDR 的 AXI Slave

开启一个 PS 侧 AXI slave 口，让 PL master 能访问 PS DDR。

```
S_AXI_HP0_FPD
```

作用：

```
HLS IP 的 m_axi_gmem* 通过这个端口读写 PS DDR
```



5.3 添加自定义 HLS IP

Block Design 中点击 `+`，搜索并添加 HLS 导出的 IP，例如：

```
rmsnorm_layer0_prefill
```

5.4 添加 AXI SmartConnect

使用两个 SmartConnect，把控制通道和数据通道分开。
添加控制通道 SmartConnect：

```
axi_smc_ctrl
```

用途：

```
PS M_AXI_HPM0_FPD -> HLS IP s_axi_control
```

添加数据通道 SmartConnect：

```
axi_smc_mem
```

用途：

```
HLS IP m_axi_gmem* -> PS S_AXI_HPC0_FPD / S_AXI_HP0_FPD -> DDR
```

RMSNorm 有 3 个 m_axi 端口，所以 `axi_smc_mem` 至少需要：

```
Number of Slave Interfaces  = 3
Number of Master Interfaces = 1
```

内存通道是：

```
rmsnorm_layer0_prefill_0/m_axi_gmem0 -> axi_smc_mem/S00_AXI
rmsnorm_layer0_prefill_0/m_axi_gmem1 -> axi_smc_mem/S01_AXI
rmsnorm_layer0_prefill_0/m_axi_gmem2 -> axi_smc_mem/S02_AXI
axi_smc_mem/M00_AXI                  -> zynq_ultra_ps_e_0/S_AXI_HPC0_FPD
```

控制通道是：

```
zynq_ultra_ps_e_0/M_AXI_HPM0_FPD     -> axi_smc_ctrl/S00_AXI
axi_smc_ctrl/M00_AXI                 -> rmsnorm_layer0_prefill_0/s_axi_control
```

5.5 添加 Processor System Reset

添加：

```
Processor System Reset
```

实例名通常是：

```
rst_ps8_0_100M
```

连接：

```
zynq_ultra_ps_e_0/pl_clk0      -> rst_ps8_0_100M/slowest_sync_clk
zynq_ultra_ps_e_0/pl_resetn0   -> rst_ps8_0_100M/ext_reset_in
```

然后：

```
rst_ps8_0_100M/peripheral_aresetn -> HLS IP ap_rst_n
rst_ps8_0_100M/peripheral_aresetn -> SmartConnect aresetn
```

时钟统一连接：

```
zynq_ultra_ps_e_0/pl_clk0 -> HLS IP ap_clk
zynq_ultra_ps_e_0/pl_clk0 -> SmartConnect aclk
zynq_ultra_ps_e_0/pl_clk0 -> PS AXI port clocks
```

6. Address Editor 地址分配

打开：

```
Window
  -> Address Editor
```

6.1 给 HLS IP 控制寄存器分配地址

给 `s_axi_control` 分配一个 PS 可访问地址。例如：

```
Master: zynq_ultra_ps_e_0/Data
Slave : rmsnorm_layer0_prefill_0/s_axi_control
Base  : 0xA0000000
Range : 64K
```

实际地址不必固定为 `0xA0000000`，以 Vivado 自动分配和 Vitis 生成的 `xparameters.h` 为准。

Vitis 中最终会出现类似宏：

```
XPAR_RMSNORM_LAYER0_PREFILL_0_BASEADDR
XPAR_XRMSNORM_LAYER0_PREFILL_0_BASEADDR
```

名字以实际工程为准。

6.2 给 HLS IP m_axi 口分配 DDR 地址空间

确认 `m_axi_gmem0/1/2` 可以访问 PS DDR。Address Editor 里应把这些 master 映射到 DDR segment，例如：

```
zynq_ultra_ps_e_0/DDR_LOW
psu_ddr_0
C0_DDR_LOW
```

```
HLS IP m_axi master 的目标必须是 PS DDR
不能悬空
不能只分配了 control 地址而没有 DDR 地址
```

7. Validate、生成 HDL Wrapper 和 Bitstream

7.1 Validate Design

点击：

```
Validate Design
```

7.2 Generate output products and Create HDL Wrapper

Sources 窗口中右键 Block Design：

```
Generate output products
Create HDL Wrapper
```

选择：

```
Let Vivado manage wrapper and auto-update
```

7.3 Generate Bitstream

直接点击：

```
Generate Bitstream
```

8. 导出硬件平台 XSA

Bitstream 生成成功后：

```
File
  -> Export
  -> Export Hardware
```

选择：

```
Include bitstream: 勾选
```

导出文件：

```
system_wrapper.xsa
```

9. Vitis IDE 创建 Platform 工程

打开 Vitis，选择 workspace，

9.1 New Platform Project

Vitis GUI：

```
File
  -> New
  -> Platform Project
```

选择：

```
Create from hardware specification (XSA)
```

导入刚才 Vivado 导出的：

```
system_wrapper.xsa
```

处理器选择：

```
psu_cortexa53_0
```

OS 选择：

```
standalone
```

Architecture：

```
64-bit
```

9.2 Build Platform

创建后右键 platform：

```
Build Project
```

确认 BSP 生成成功。BSP 中应能看到：

```
standalone
xilffs / lwip 等可选库
HLS IP driver
```

HLS IP driver 对当前 RMSNorm 通常包含：

```
xrmsnorm_layer0_prefill.h
xrmsnorm_layer0_prefill.c
```

10. 创建 lwIP 网络 Application

打开 BSP/domain settings。
使用静态 IP：

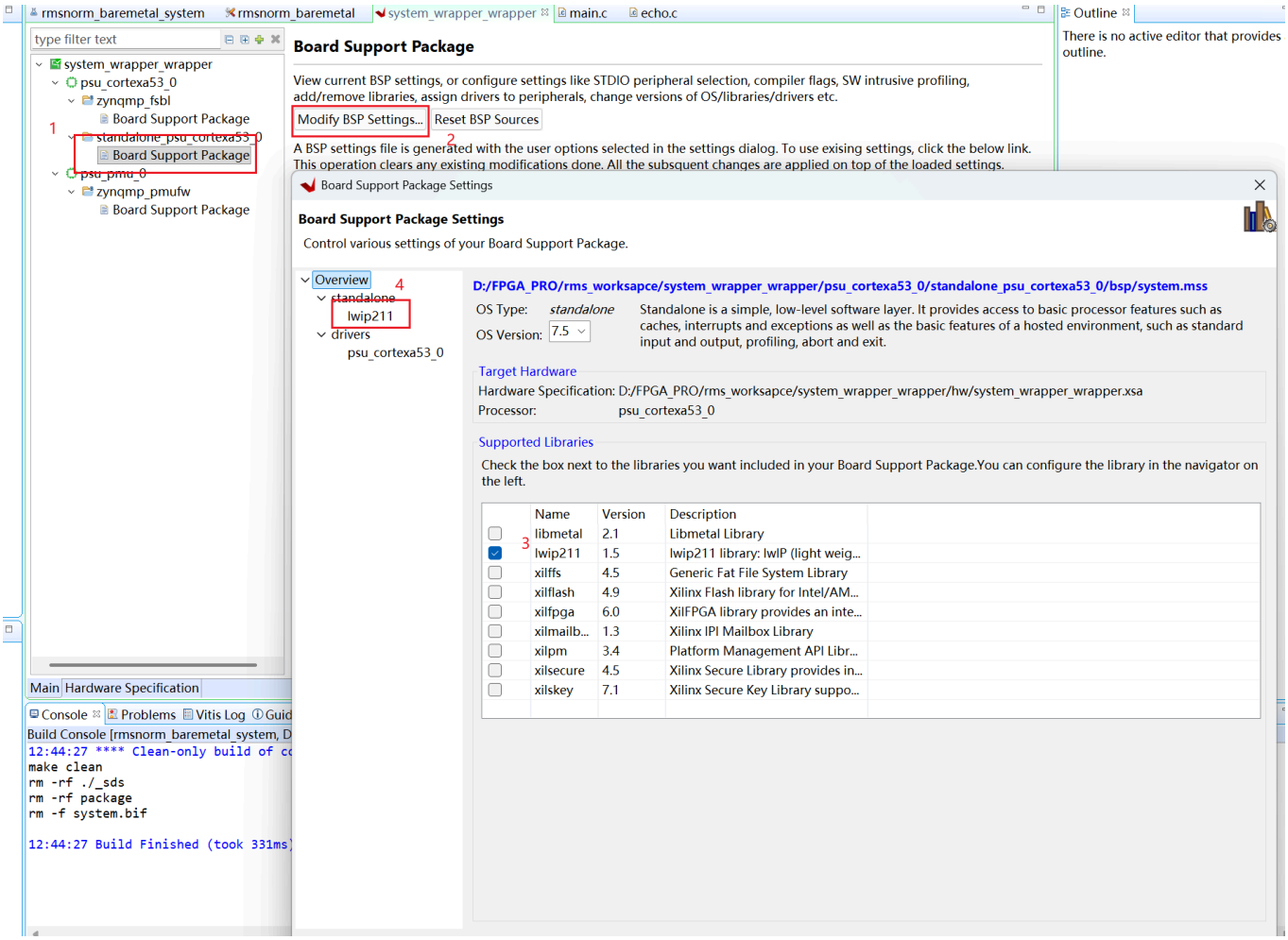
```
lwip_dhcp = false
```

Vitis lwIP echo server 默认静态配置是：

```
Board IP: 192.168.8.190
Netmask : 255.255.255.0
Gateway : 192.168.8.1
```

把 lwIP buffer 调大：

```
tcp_snd_buf        = 32768
tcp_wnd            = 32768
memp_num_tcp_seg   = 512
mem_size           >= 262144
pbuf_pool_size     >= 256
```



11. Vitis 下载和运行

11.1 硬件连接

确认：

```
板卡上电
JTAG 已连接
UART 已连接并打开串口终端
跑 lwIP，Ethernet 已连接
```

串口配置：

```
115200 8N1
```

11.2 Run Configuration

右键 app：

```
Run As
  -> Run Configurations
```

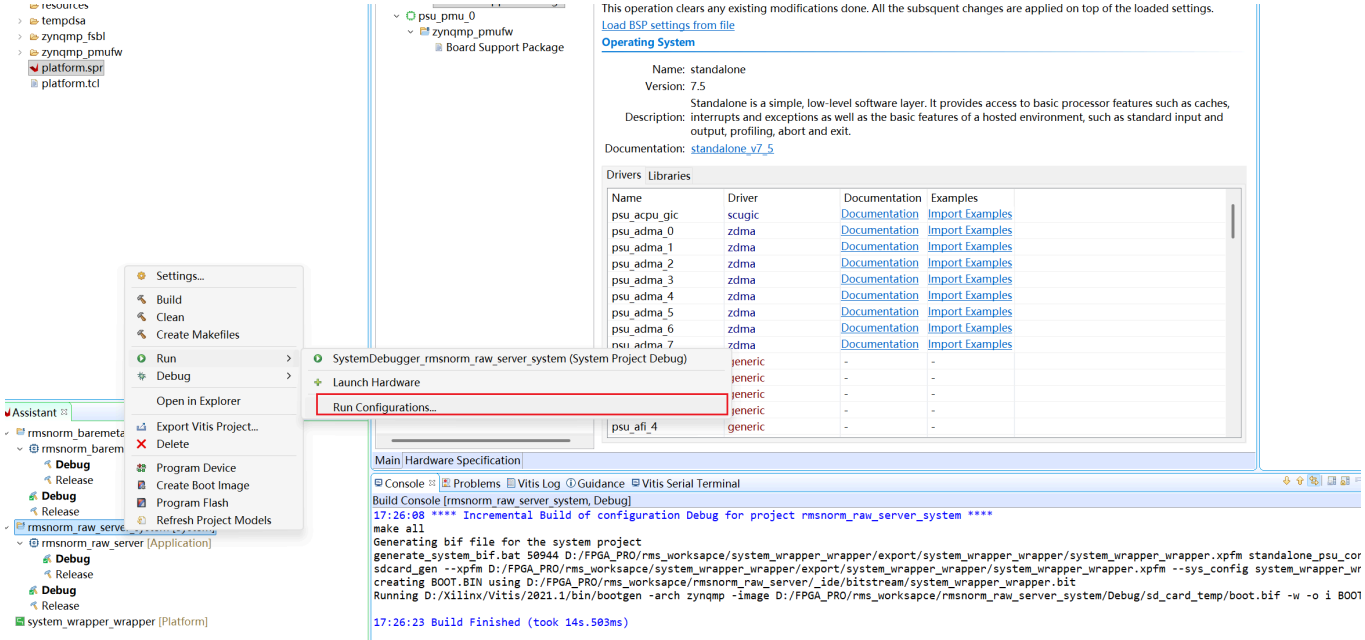
选择或创建：

```
Single Application Debug
```

Target processor：

```
psu_cortexa53_0
```

```
Program FPGA            勾选
Initialize using FSBL   勾选
Use FSBL flow           勾选
Reset entire system     勾选
```



13. Orin 侧联调

把 Orin 和 AXU9EGB PS Ethernet 接到同一个交换机。

```
python qwen25_0p5b_python_reference/qwen2_generate.py \
  --model-path /home/nvidia/models/Qwen2.5-0.5B-Instruct \
  --device cuda:0 \
  --dtype float16 \
  --max-new-tokens 1 \
  --rmsnorm-backend raw-ethernet \
  --rmsnorm-host 192.168.1.10 \
  --rmsnorm-port 50052 \
  --rmsnorm-timeout 30 \
  --output-json raw_generate_1tok.json
```

1 token 通过后，再跑更多 token：

```
python qwen25_0p5b_python_reference/qwen2_generate.py \
  --model-path /home/nvidia/models/Qwen2.5-0.5B-Instruct \
  --device cuda:0 \
  --dtype float16 \
  --max-new-tokens 8 \
  --rmsnorm-backend raw-ethernet \
  --rmsnorm-host 192.168.1.10 \
  --rmsnorm-port 50052 \
  --rmsnorm-timeout 30 \
  --output-json raw_generate_8tok.json
```